project. Our goal is to give you the tools and context to decide what's best for your own situation.

Integration specs are harder to write than unit specs, and they run more slowly. Therefore, we prefer to write fewer integration specs and more unit specs.

To make this arrangement possible, we need to keep our interfaces to external resources small and well defined. That way, we only have to write a few integration specs for these interfaces. Our faster unit specs can easily substitute fake versions of the interfaces.

Unfortunately, the general advice to favor unit specs isn't always easy to carry out in the real world. For many projects, the highest-value specs are also the ones that cost the most.

For instance, the ability to refactor your application logic is extremely valuable. End-to-end acceptance specs provide the best refactoring support. Because they only use your code's public interfaces, they don't depend on implementation details.

Of course, unit specs help with low-level refactoring such as reimplementing a specific method. But they won't support larger refactoring efforts such as removing a class entirely and distributing its logic elsewhere.

Acceptance specs' refactoring support comes at a price. They are more difficult to write, more brittle, and slower than other specs. But they provide so much confidence that it's important to have them—we just write very few of them, focus on the happy path, and don't use them for exhaustive coverage of conditional branches.

In the next part of this book, you're going to write an app from scratch, using all three of these types of specs along the way. As you reach each stage of the project, think about your goal for the spec you're about to write. That will guide you naturally toward which type to use.